

Phase-4

Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

```
src > models > User.js >
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcrypt');
3
4 const userSchema = new mongoose.Schema(
5   {
6     name: {
7       type: String,
8       required: [true, 'Name is required'],
9       trim: true,
10    },
11    email: {
12      type: String,
13      required: [true, 'Email is required'],
14      unique: true,
15      lowercase: true,
16      trim: true,
17      match: [/^\s*@.*\s$/, 'Please provide a valid email address'],
18    },
19    password: {
20      type: String,
21      required: [true, 'Password is required'],
22      minlength: [6, 'Password must be at least 6 characters long'],
23    },
24  },
25  {
26    timestamps: true,
27  }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

```
src > models > FAQ.js >
1 const mongoose = require('mongoose');
2
3 const faqSchema = new mongoose.Schema(
4   {
5     question: {
6       type: String,
7       required: [true, 'Question is required'],
8       trim: true,
9     },
10    answer: {
11      type: String,
12      required: [true, 'Answer is required'],
13      trim: true,
14    },
15    category: {
16      type: String,
17      required: [true, 'Category is required'],
18      enum: {
19        values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20        message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking, ...',
21      },
22      default: 'General',
23    },
24    createdBy: {
25      type: mongoose.Schema.Types.ObjectId,
26      ref: 'User',
27      required: [true, 'Creator user reference is required'],
28    },
29  },
30  {
31    timestamps: true,
32  }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
```